

UNIT-V

STRUCTURAL PATTERN PART-II

DECORATOR PATTERN :

Intent :

Attach additional responsibilities to an object dynamically.

Decorator provide a flexible alternative to sub classing for extending functionality.

Also known As :

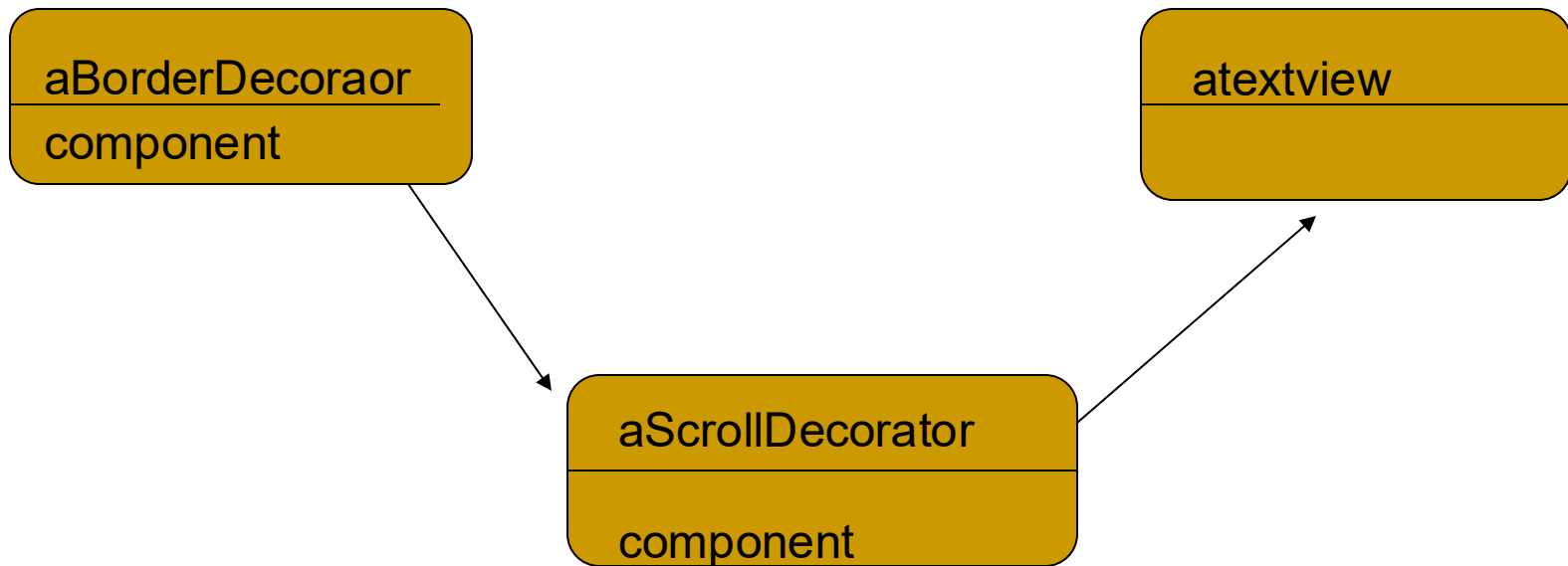
Wrapper

motivation :

- sometimes we want to add responsibilities to individual objects not to an entire class
- for e.g;- a graphical user interface toolkit should let u add properties like borders or scrollbars to any user interface component.
- one way to add responsibilities is with inheritance inheriting from another class puts a border around every subclass instance.
- this is inflexible , because the choice of border is made statically.
- a more flexible approach is to enclose the component in another object that adds the border. the enclosing object is called **decorator**.

- the decorator conforms to the interface of the component it decorates so that its presence is transparent to the components client.
- the decorator forwards requests to the component & may perform additional actions before or after forwarding.
- transparency lets to nest decorators recursively , allowing to add unlimited no.of additional responsibilities.
- for e.g.- a **textView** object that displays text in a window has no scrollbars by default , we can use a **scrollDecorator** to add them. if we also want to add border around the TextView we use **borderDecorator**.
- simply compose the decorators with the textview to produce the desired result.

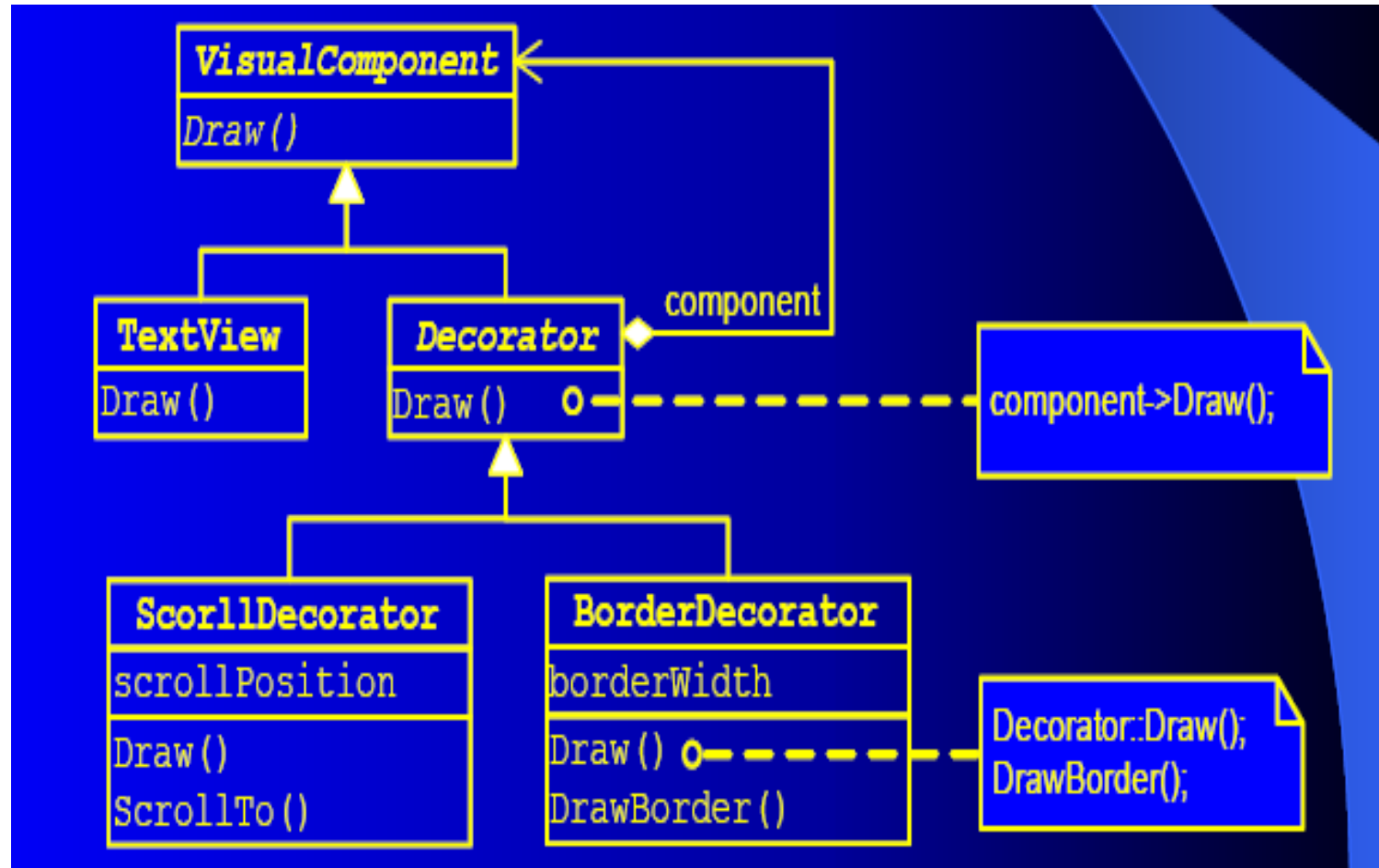
how to compose a textview object with borderdecorator & scrollDecorator.



- scrollDecorator & BorderDecorator classes are subclasses of Decorator , an abstract class for visual components that decorate other visual components.

fig :-

- visual components is the abstract class for visual objects, it defines drawing & event handling interface.
- decorator class simply forwards draw requests to its component, decorator subclasses are free to add operations for specific functionality.



collaborations :

decorator forwards requests to its component object.

consequences :

➤ more flexibility than static inheritance : -

1. decorator pattern provides a more flexible way to add responsibilities to objects than with static inheritance (multiple).
2. with decorators , responsibilities can be added & removed at runtime simply by attaching & detaching them.
3. in contrast , inheritance requires creating a new class for additional responsibility this results in many classes & increases the complexity of a system.

4. decorators also make it easy to add a property twice for
e.g.- to add a double border, simply attach two border
decorators (inheriting twice is error-prune).

- avoids feature-laden classes high up in the hierarchy
instead of trying to support all features , define a simple
class & add functionality incrementally with decorator objects.

liabilities :

1. a decorator & its component aren't identical :- a decorator
acts as a transparent enclosure.

a lots of little objects :- usage of decorator often results in lots of little objects that all look alike.

implementation :

issues to be considered when applying the decorator pattern.

1. interface conformance :

a decorator object's interface must conform to the interface of the component it decorates.

2. omitting the abstract decorator class :

no need to define an abstract decorator class when u only need to add one responsibility.

3. keeping component classes lightweight : to ensure a conforming interface , components & decorators must descend from a common component class & it is important to keep this common class lightweight i.e , it should focus on defining an interface , not on storing data.

known uses :

interviews , ET++, objectworks\smalltalk class library.

debuggingGlyph from Interviews and the passivityWrapper from parcplace smalltalk.

related patterns :

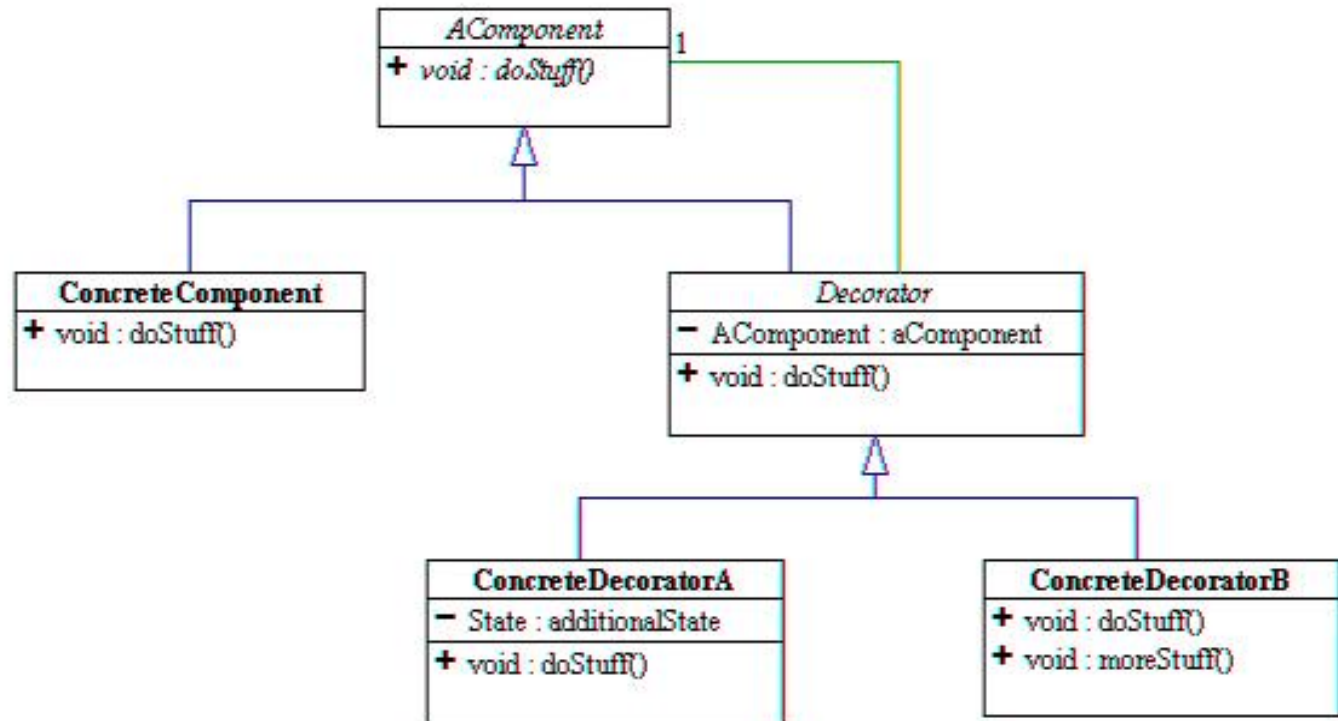
adapter : a decorator is different from an adapter in that a decorator only changes an object's responsibilities,

not its interface; an adapter will give an object a completely new interface.

composite : decorator can be viewed as a degenerate composite with one component. however a decorator adds additional responsibilities-----it is'nt intended for object aggregation.

strategy : a decorator lets u change the skin of an object, a strategy lets u change the guts.

Structure :



Participants :

Component (VisualComponent)

- defines the interface for objects that can have responsibilities added to them dynamically.

ConcreteComponent (TextView)

- defines an object to which additional responsibilities can be attached.

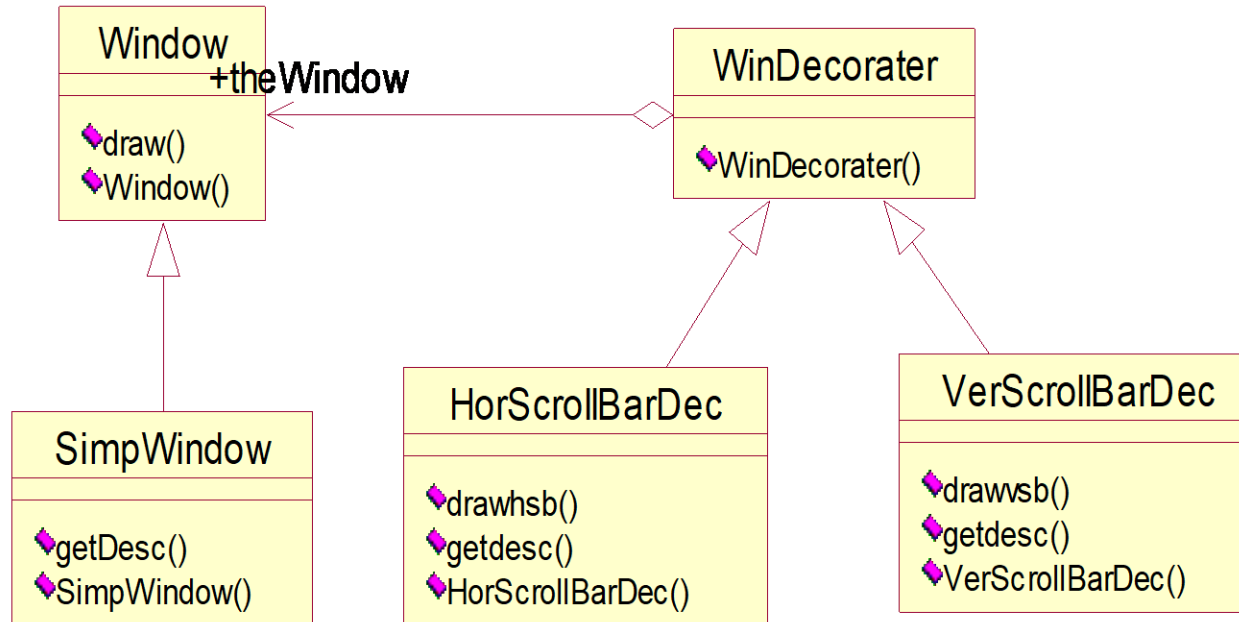
Decorator

- maintains a reference to a Component object and defines an interface that conforms to component's interface.

ConcreteDecorator (BorderDecorator)

- adds responsibilities to the component.

Sample code :



**abstract public class WinDecorater extends
Window**

```
{  
    public Window decoratedWindow;  
    // public Window theWindow;  
  
    public WinDecorater(Window decoratedWindow)  
    {  
        this.decoratedWindow=decoratedWindow;  
    }  
}
```

```
public class Window
```

```
{
```

```
    //public WinDecorater theWinDecorater;
```

```
    public Window()
```

```
    {
```

```
    }
```

```
    public void draw()
```

```
    {
```

```
    }
```

```
    public String getdesc()
```

```
{
```

```
    return "null";
```

```
}
```

```
}
```

```
public class SimpWindow extends Window
{
    public SimpWindow()
    {

    }
    public String getDesc()
    {
        return "Simple Window";
    }
    public void draw()
    {
    }
}
```

```
public class VerScrollBarDec extends WinDecorater
{
    public VerScrollBarDec(Window decoratedWindow)
    {
        super(decoratedWindow);
    }
    public void drawvsb()
    {
    }
    public String getdesc()
    {
        return decoratedWindow.getdesc()+"including vertical scroll bars";
    }
    public void draw()
    {
        drawvsb();
        decoratedWindow.draw();
    }
}
```

```
public class HorScrollBarDec extends WinDecorater
{
    public HorScrollBarDec(Window decoratedWindow)
    {
        super(decoratedWindow);
    }
    public void drawhsb()
    {
    }
    public String getdesc()
    {
        return decoratedWindow.getdesc()+"including horizontal scroll bars";
    }
    public void draw()
    {
        drawhsb();
        decoratedWindow.draw();
    }
}
```

public class decorator

```
{  
    public static void main(String args[])  
    {  
Window decoratedWindow  
=new HorScrollBarDec( new VerScrollBarDec(new  
    SimpWindow()));  
        System.out.println(decoratedWindow.getdesc());  
    }  
}
```

Façade pattern

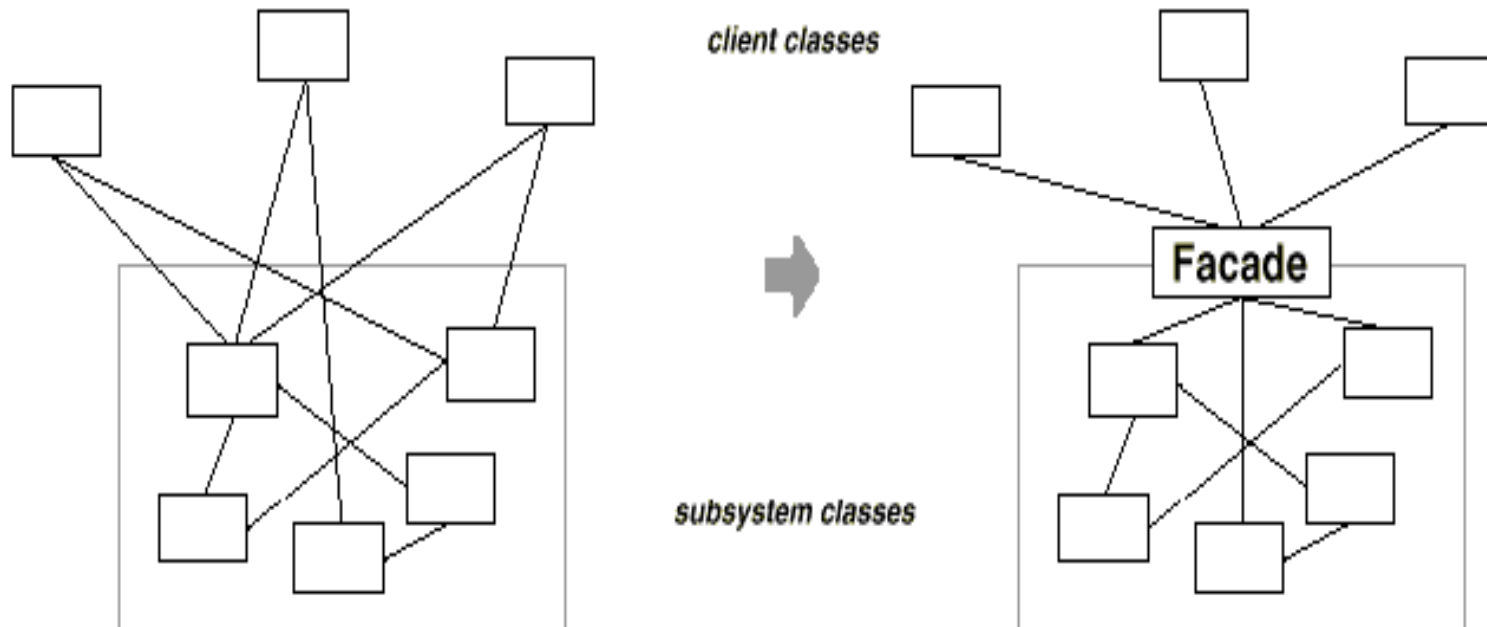
Intent :

Provide a unified interface to a set of interfaces in a subsystem. Façade defines a higher-level interface that makes the subsystem easier to use.

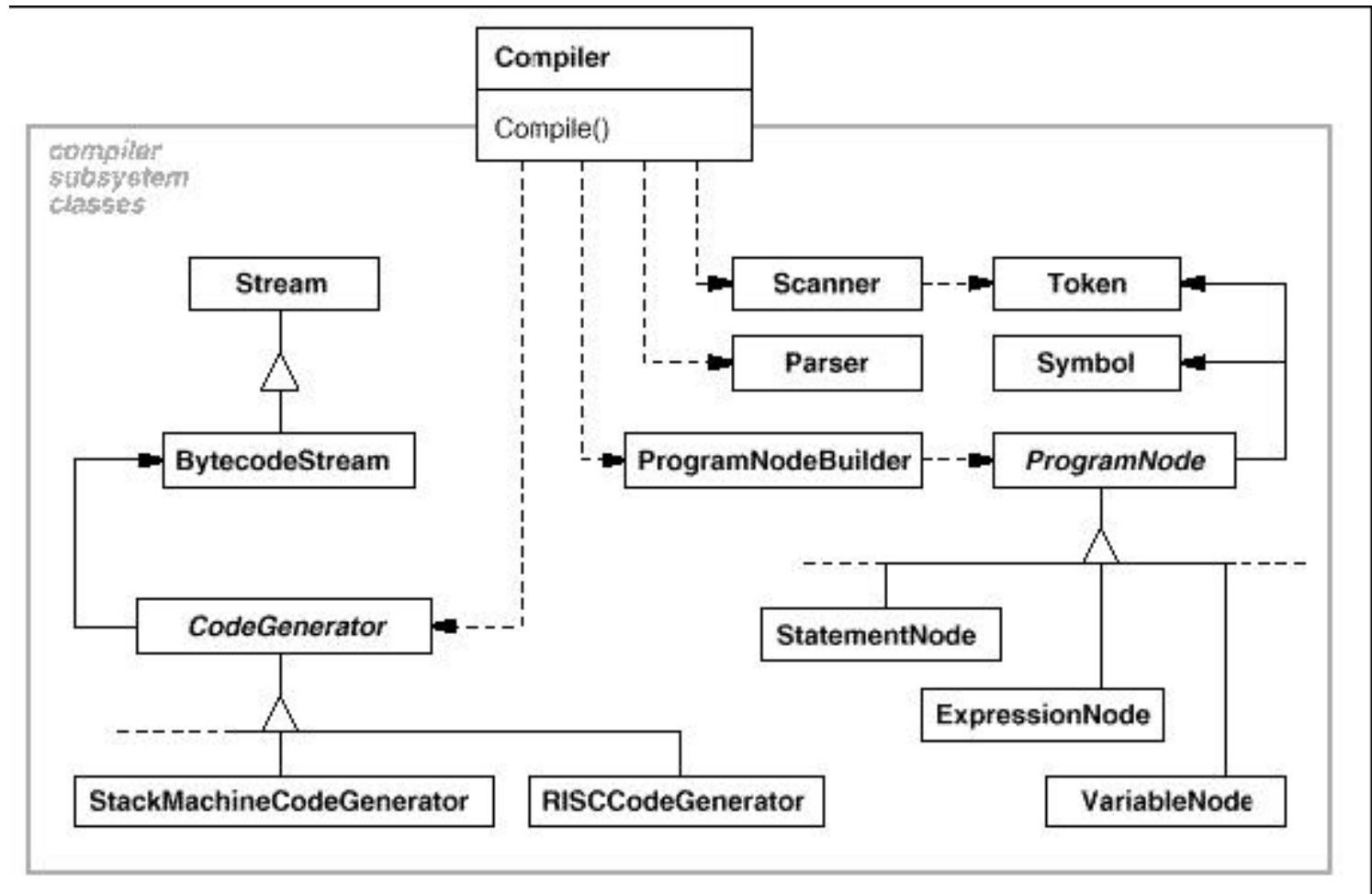
Motivation :

- Structuring a system into subsystems helps reduce complexity.
- Subsystems are groups of classes, or groups of classes and other subsystems.
- The interface exposed by the classes in a subsystem or set of subsystems can become quite complex

- One way to reduce this complexity is to introduce a facade object that provides a single, simplified interface to the more general facilities of a subsystem.



For e.g.-compiler subsystem



- This subsystem contains classes such as Scanner, Parser, ProgramNode, BytecodeStream, and ProgramNodeBuilder that implement the compiler.
- Most clients of a compiler generally don't care about details like parsing and code generation; they merely want to compile some code.
- To provide a higher-level interface that can shield clients from these classes, the compiler subsystem includes a compiler class.
- This class defines a unified interface to the compiler's functionality.
- The compiler class acts as a FAÇADE : it offers clients a simple interface to the compiler subsystem.

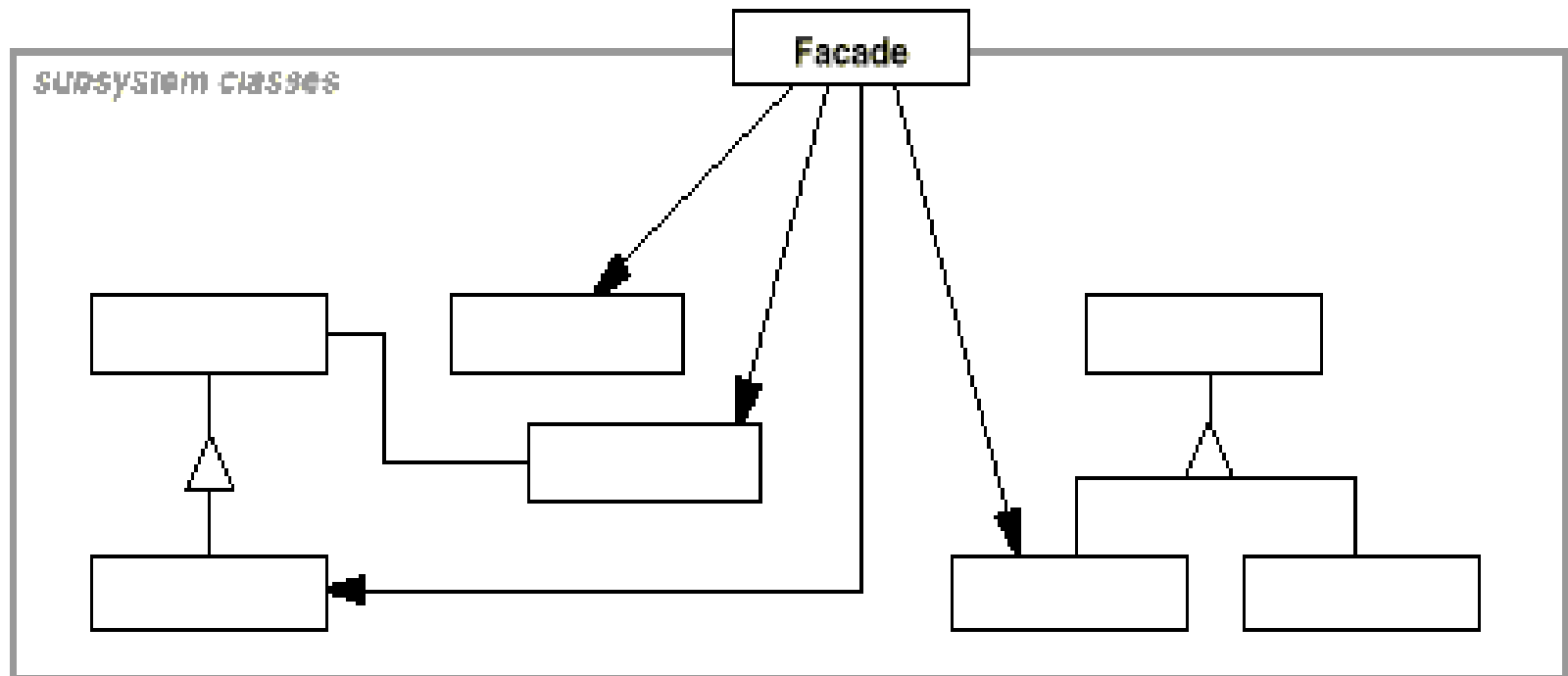
-
- It glues together the classes that implement compiler functionality without hiding them completely.

Applicability :

Use the Facade pattern:

- To provide a simple interface to a complex subsystem. This interface is good enough for most clients; more sophisticated clients can look beyond the facade.
- To decouple the classes of the subsystem from its clients and other subsystems, thereby promoting subsystem independence and portability.
- To layer subsystems, then use façade to define an entry point to each subsystem level.

Structure :



Participants :

Façade (compiler)

- Knows which subsystem classes are responsible for a request.
- Delegates client requests to appropriate subsystem objects.

Subsystem classes (scanner, parser, programNode etc)

- Implement subsystem functionality
- Handle work assigned by the façade object.
- Have no knowledge of the façade; that is , they keep no references to it.

Collaborations :

Clients communicate with the subsystem by sending requests to façade, which forwards them to the appropriate subsystem object (s).

-
- Clients that use the façade don't have to access its subsystem objects directly.

Consequences :

Benefits

- It hides the implementation of the subsystem from clients, making the subsystem easier to use
- It promotes weak coupling between the subsystem and its clients. This allows you to change the classes the comprise the subsystem without affecting the clients.
- It reduces compilation dependencies in large software systems classes!

- It simplifies porting systems to other platforms, because it's less likely that building one subsystem requires building all others
- It does not prevent sophisticated clients from accessing the underlying classes
- Note that Facade does not add any functionality, it just simplifies interfaces
- It does not prevent clients from accessing the underlying classes!

Flyweight pattern

Intent :

use sharing to support large numbers of fine-grained objects efficiently. (A Flyweight is an object that minimizes memory occupation by sharing as much data as possible with other similar objects)

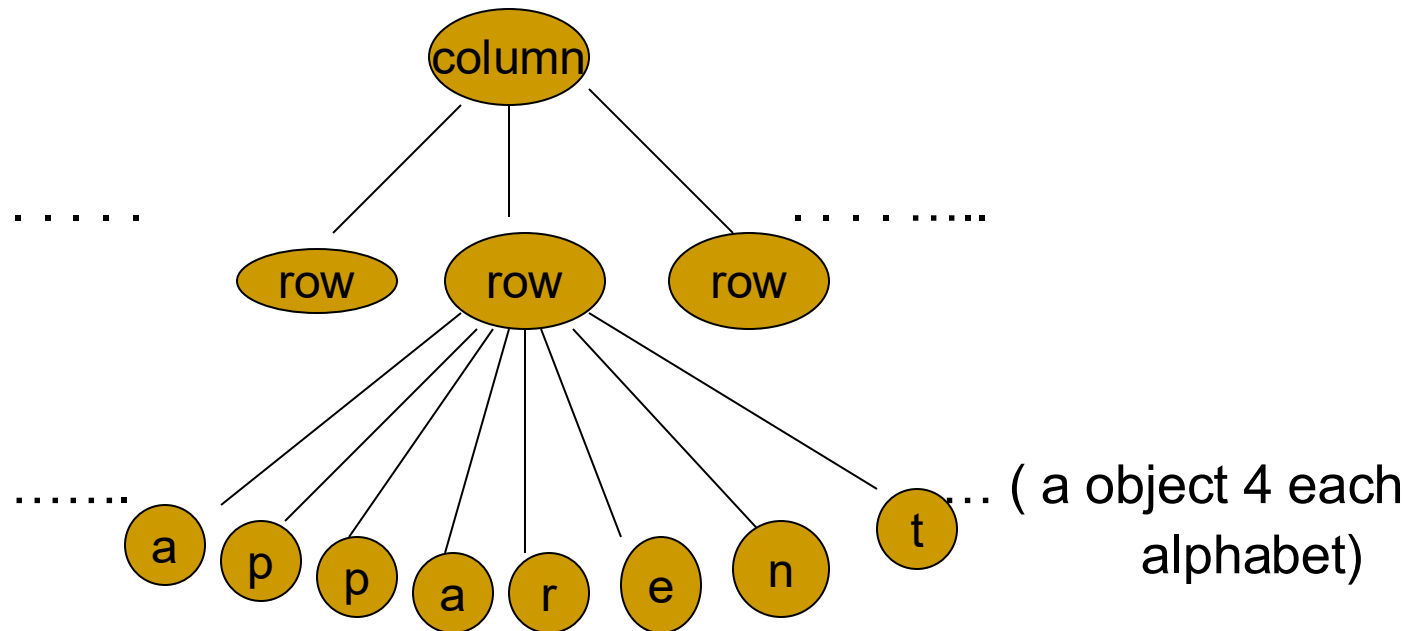
Motivation :

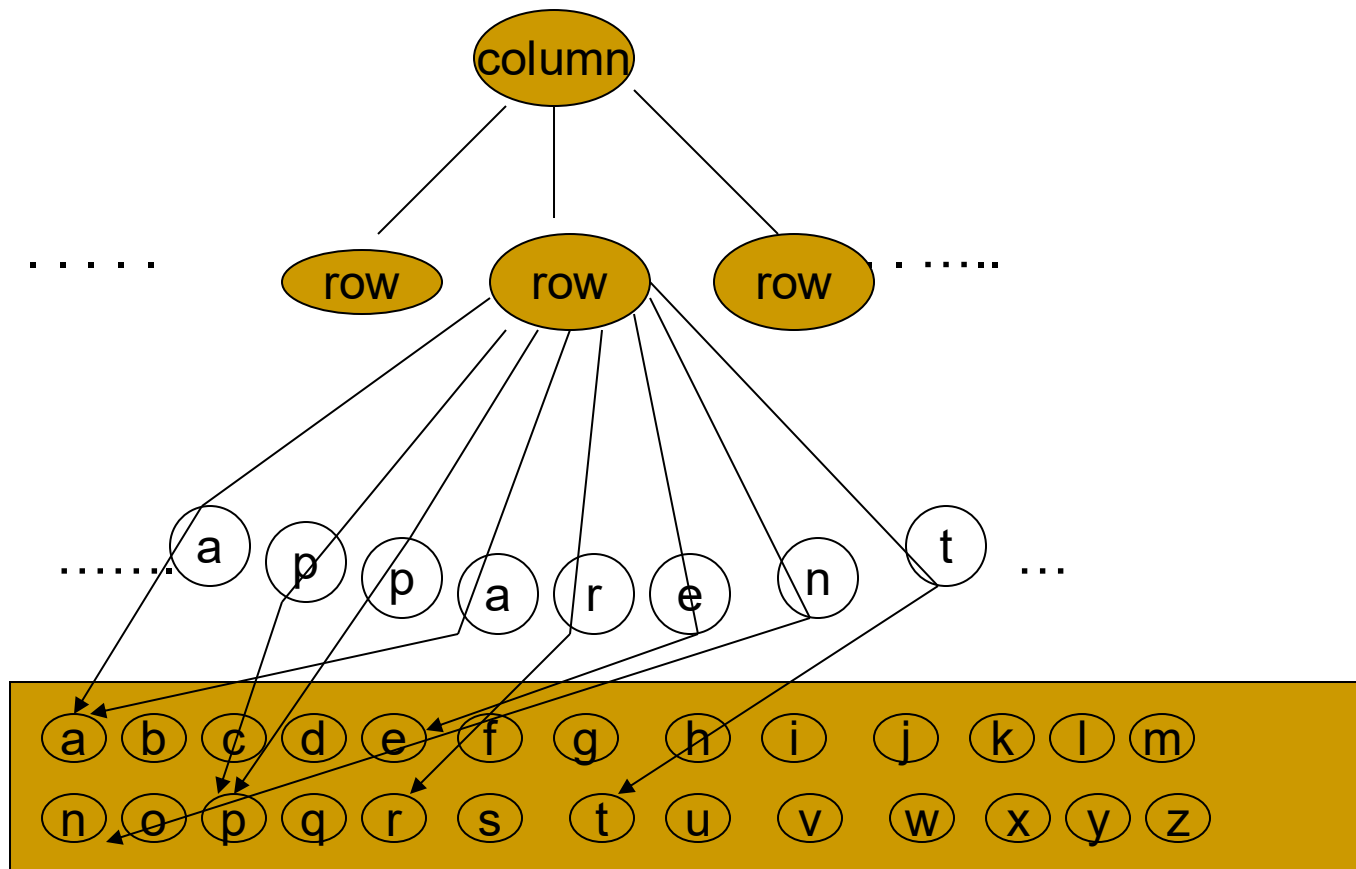
- For example, when designing a word processor application, you might create an object for each character typed. Each Character object might contain information such as the font face, size and weight of each character.

- The problem here is that a lengthy document might contain tens of thousands of characters, and objects which results in memory wastage
- The Flyweight pattern addresses the problem by creating a new object to store such information, which is shared by all characters with the same formatting.
- For e.g.- if a document contains ten-thousand words, with 800 characters in Bold Times-New-Roman, these 800 characters would contain a reference to a flyweight object that stores their common formatting information.
- The key here is that you only store the information once, so memory consumption is greatly reduced.

- a *flyweight* is a shared object that be used in multiple contexts simultaneously.
- Flyweight acts as an independent object in each context.
- The key concept is the distinction between *intrinsic* & *extrinsic* state.
- Intrinsic state is stored in the flyweight , it consists of information that's independent of the flyweight context, there by making it sharable.
- Extrinsic state depends on and varies with the flyweight's context and therefore can't be shared. Client objects are responsible for passing extrinsic state to the flyweight when it needs.

- For e.g.-a document editor can create a flyweight for each letter of the alphabet.
- Each flyweight stores a character code, but its coordinate position in the document and its typographic style can be determined from the text layout algorithms wherever the character appears.
- The character code is intrinsic state, while other information is extrinsic .



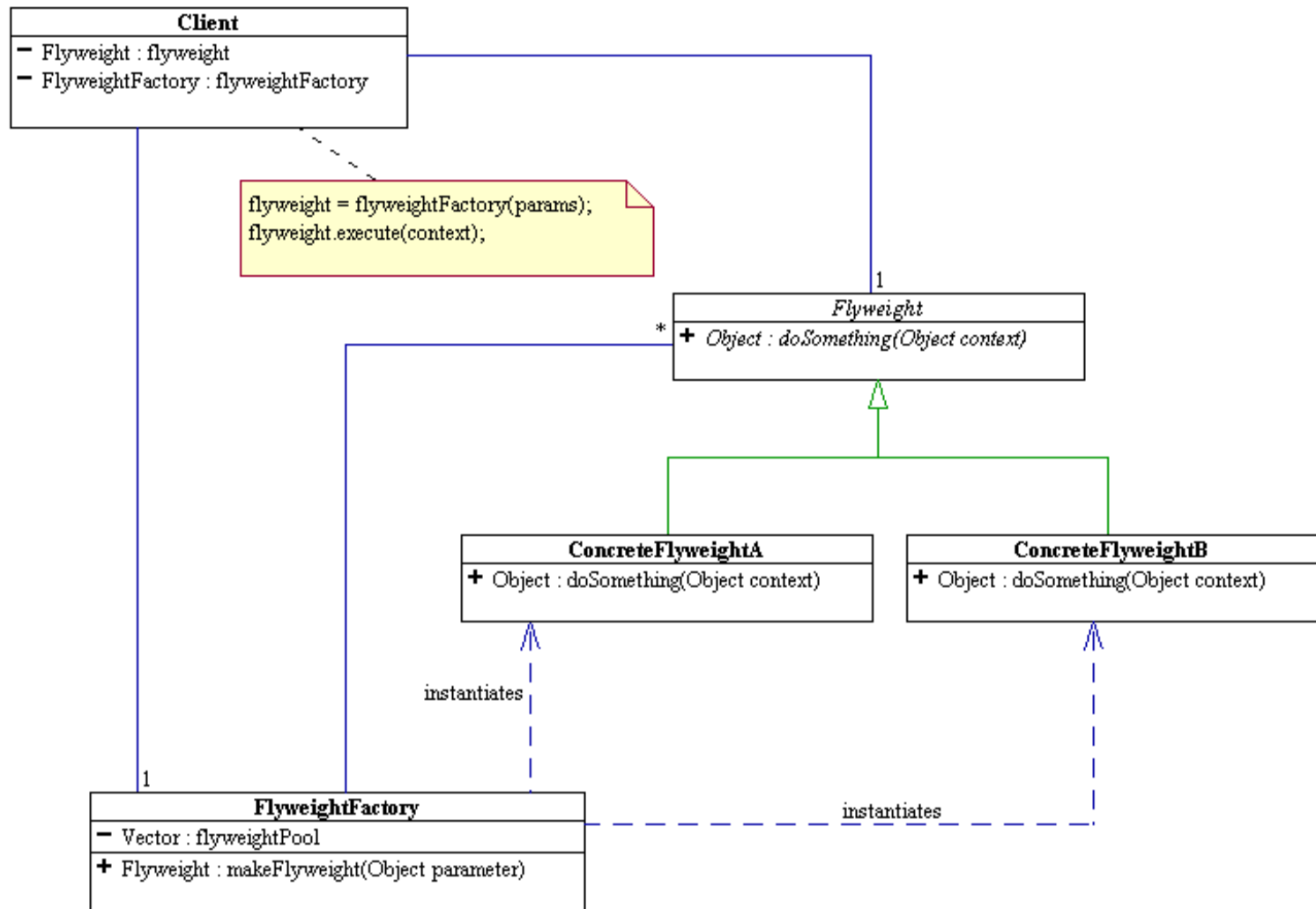


flyweight pool

Applicability :

Apply the flyweight pattern when all of the following r true :

- an application uses a large no.of objects.
- Storage costs r high because of the sheer quantity of objects.
- Most object state can b made extrinsic.
- Many group of objects may be replaced by relatively few shared objects once extrinsic state is removed.



Object diagram :

Participants

Flyweight (glyph)

declares an interface through which flyweights can receive and act on extrinsic state.

concreteFlyweight (character)

implements the flyweight interface and adds storage for intrinsic state, it shld be sharable.

Unsharedconcreteflyweight (row,column)

not all flyweight subclasses need to be shared.

Flyweightfactory

creates and manages flyweight objects.

ensures that flyweights are shared properly

Client

maintains a reference to flyweight (s)

computes or stores the extrinsic state of flyweight (s)

Collaborations :

Clients should not instantiate concreteflyweight directly. Clients must obtain concreteflyweight objects exclusively from flyweight factory object to ensure they r shared properly.

Consequences :

- The more flyweights r shared , the greater the storage savings.
- Flyweight pattern is often combined with the composite pattern to represent a hierarchical structure as a graph with shared leaf nodes

Implementation :

issues when implementing the flyweight pattern :

Removing extrinsic state :

- in document editor e.g.- we can store a map of typographic information in a separate structure rather than store the font and type style with each character object.
- The map keep tracks of runs of characters with the same typographic attributes.
- When a character draws itself, it receives its typographic attributes .

Managing shared objects :

- Because objects r shared , clients should not instantiate them directly.
- flyweight factory lets clients locate a particular flyweight .

Known uses :

Interviews 3. 0, ET++ uses flyweights to support look and feel independence.

Related patterns :

Composite pattern -----for hierarchical structure .
State and strategy objects are implemented as flyweights.

```
class TeaOrderContext
{
    int tableNumber;
    TeaOrderContext(int tableNumber)
    {
        this.tableNumber = tableNumber;
    }

    public int getTable()
    {
        return this.tableNumber;
    }
}
```

```
abstract class TeaOrder
{
    public abstract void serveTea(TeaOrderContext teaOrderContext);
}
class TeaFlavor extends TeaOrder
{
    String teaFlavor;

    TeaFlavor(String teaFlavor)
    {
        this.teaFlavor = teaFlavor;
    }

    public String getFlavor()
    {
        return this.teaFlavor;
    }
}
```

```
public void serveTea(TeaOrderContext teaOrderContext)
{
    System.out.println("Serving tea flavor " +
        teaFlavor +
        " to table number " +
        teaOrderContext.getTable());
}
}
class TeaFlavorFactory
{
    TeaFlavor[] flavors = new TeaFlavor[10];
    //no more than 10 flavors can be made
    int teasMade = 0;
```

```
public TeaFlavor getTeaFlavor(String flavorToGet)
{
    if (teasMade > 0)
    {
        for (int i = 0; i < teasMade; i++)
        {
            if (flavorToGet.equals((flavors[i]).getFlavor()))
            {
                return flavors[i];
            }
        }
    }
    flavors[teasMade] = new TeaFlavor(flavorToGet);
    return flavors[teasMade++];
}

public int getTotalTeaFlavorsMade()
{
    return teasMade ; }
}
```

```
public class FlyweightExample
{
    static TeaFlavor[] flavors = new TeaFlavor[100];
    //the flavors ordered
    static TeaOrderContext[] tables = new TeaOrderContext[100];
    //the tables for the orders
    static int ordersMade = 0;
    static TeaFlavorFactory teaFlavorFactory;
    static void takeOrders(String flavorIn, int table)
    {
        flavors[ordersMade] = teaFlavorFactory.getTeaFlavor(flavorIn);
        tables[ordersMade++] = new TeaOrderContext(table);
    }
}
```

```
public static void main(String[] args)
{
    teaFlavorFactory = new TeaFlavorFactory();
    takeOrders("chai", 2);
    takeOrders("chai", 2);
    takeOrders("camomile", 1);
    takeOrders("camomile", 1);
    takeOrders("earl grey", 1);
    takeOrders("camomile", 897);
    takeOrders("chai", 97);
    takeOrders("chai", 97);
    takeOrders("camomile", 3);
    takeOrders("earl grey", 3);
    takeOrders("chai", 3);
    takeOrders("earl grey", 96);
    takeOrders("camomile", 552);
    takeOrders("chai", 121);
    takeOrders("earl grey", 121);
}
```

```
for (int i = 0; i < ordersMade; i++)
{
    flavors[i].serveTea(tables[i]);
}
System.out.println(" ");
System.out.println("total teaFlavor objects made: " +
    teaFlavorFactory.getTotalTeaFlavorsMade());
}
}
```

/*

OUTPUT :

Serving tea flavor earl grey to table number 3

Serving tea flavor chai to table number 3

Serving tea flavor earl grey to table number 96

Serving tea flavor camomile to table number 552

Serving tea flavor chai to table number 121

Serving tea flavor earl grey to table number 121

total teaFlavor objects made: 3

*/